

Résumé des projets réalisés durant ma licence Informatique

Au cours de ma licence Informatique à l'Université Paris-Saclay, j'ai réalisé plusieurs projets techniques me permettant d'appliquer mes connaissances en programmation, algorithmique et analyse de données.

En troisième année, j'ai choisi le parcours MIAGE afin de compléter ma formation informatique par des enseignements liés aux systèmes d'information et à l'entreprise.

Les projets présentés ci-dessous illustrent les compétences que j'ai développées durant ma formation, notamment en développement logiciel, analyse de données, programmation orientée objet et conception d'applications.

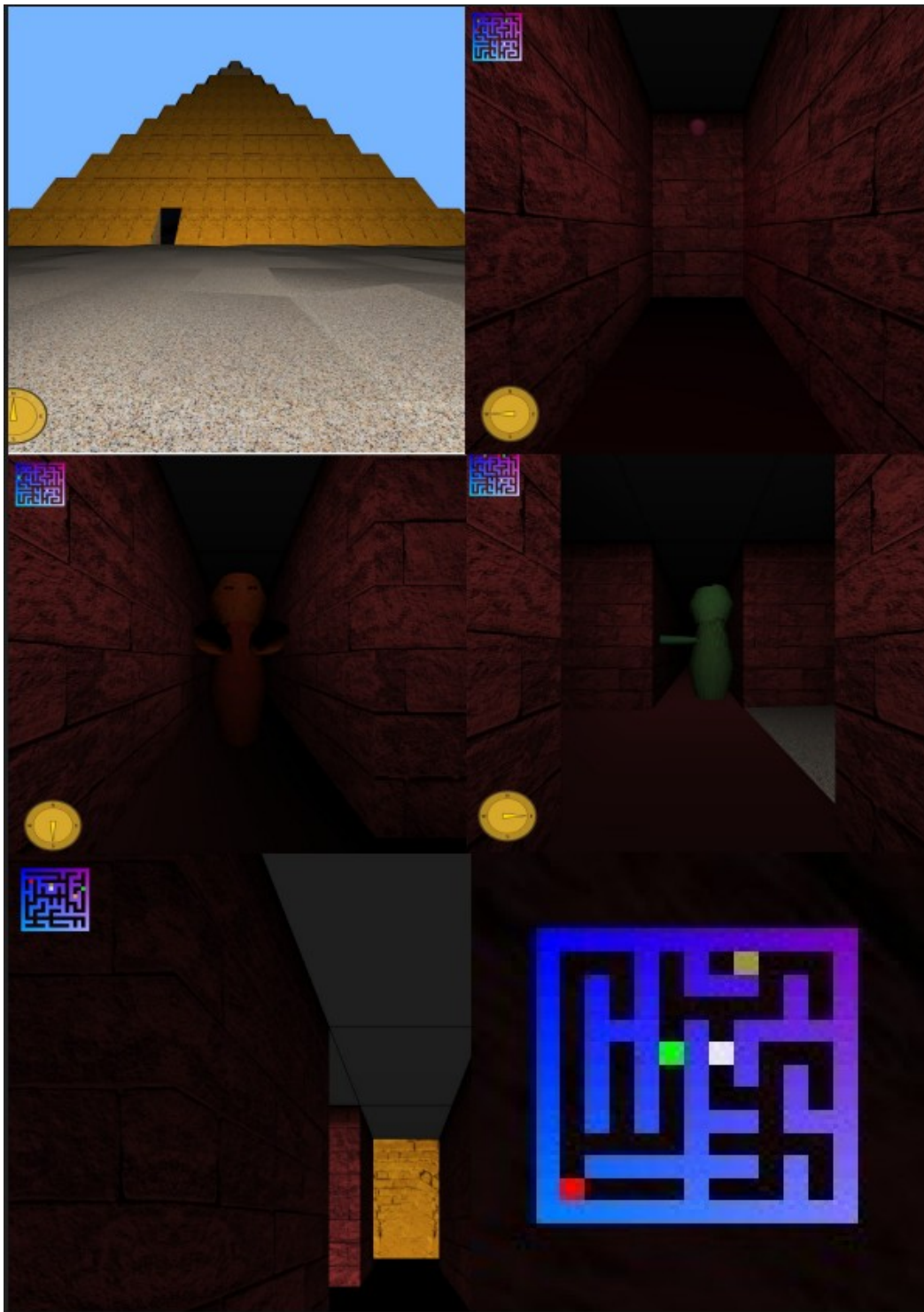
D'autres projets ont également été réalisés durant ma formation, notamment dans le cadre de travaux pratiques et de projets encadrés. Afin de respecter la limite maximale de cinq pages demandée pour ce document, seuls quelques exemples représentatifs ont été présentés.

Projet Graphique 3D (Processing : Python) : La Pyramide Mystérieuse.

Objectif : Ce projet consiste à explorer une pyramide mystérieuse composée de 5 étages de labyrinthes générés procéduralement.

Le joueur doit se frayer un chemin en évitant des momies errantes et atteindre le trésor caché au plus profond du monument pour terminer l'aventure. Vous avez à la fin des images à disposition pour illustrer le projet.

Synopsis : Dans un désert perdu d'Égypte, une ancienne pyramide vient d'être découverte. Les explorateurs racontent que quiconque y entre n'en ressort jamais. Vous incarnez un aventurier qui décide de s'y aventurer à la recherche d'un trésor légendaire. Traversez les 5 étages, évitez les momies, et trouvez le trésor doré pour espérer survivre.

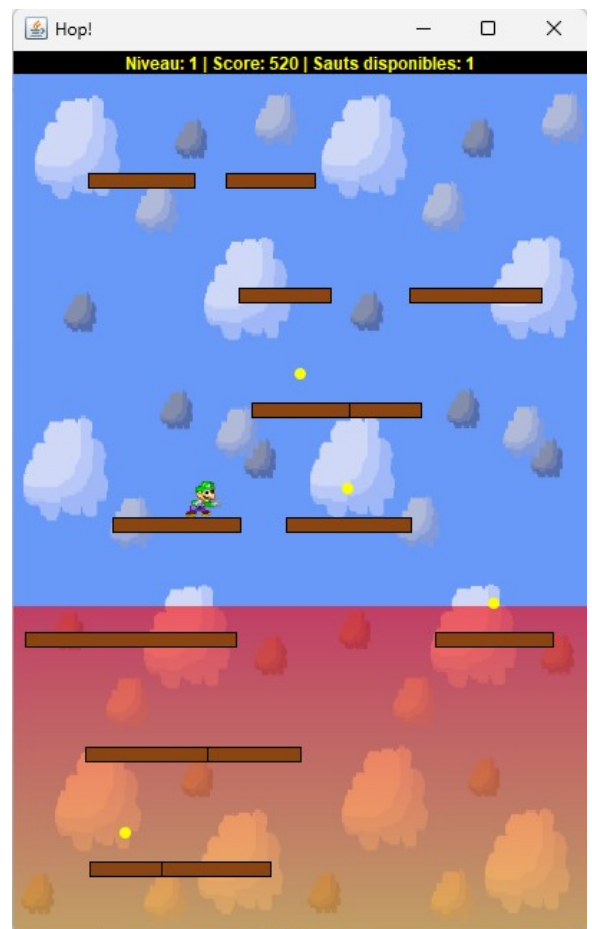
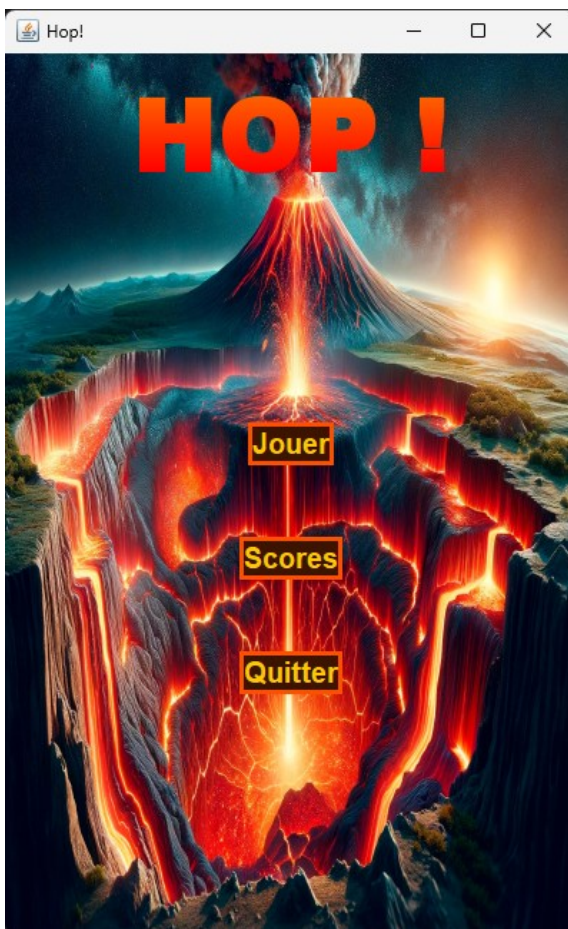


Projets Java

Plusieurs projets réalisés durant ma formation m'ont permis de mettre en pratique la programmation orientée objet en Java et d'explorer la conception d'applications interactives.

Le projet *Colt Express* consistait à développer une application complète en Java en utilisant une architecture MVC. Le projet impliquait la gestion des actions des joueurs, des déplacements, des collisions ainsi que l'affichage en temps réel des événements du jeu.

Le projet *Évasion d'un volcan*, réalisé en binôme, portait sur la conception d'un jeu vidéo. Nous avons travaillé sur la conception du gameplay, la mise en place de la logique du jeu ainsi que l'organisation du code. Ce projet nous a également permis de collaborer à l'aide d'un système de gestion de version (Git) et de présenter notre travail lors d'une soutenance orale.



Projet Data Science – Simulation évolutive et analyse de données

Dans le cadre d'un projet d'analyse de données scientifiques, je travaille actuellement sur la conception d'un modèle de simulation évolutive permettant d'étudier l'évolution de populations sur plusieurs générations. Ce projet utilise le logiciel de simulation génétique SLiM, qui permet de modéliser différents scénarios évolutifs et environnementaux.

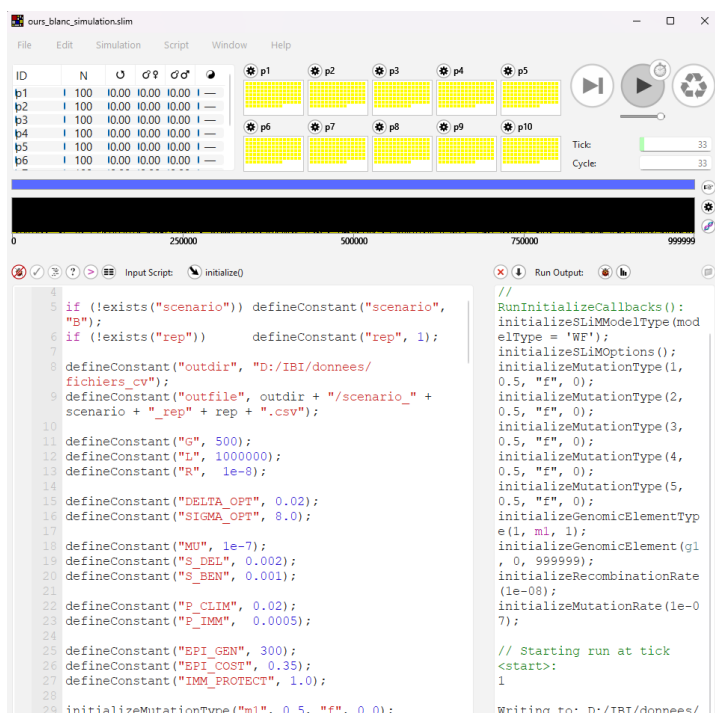
Le modèle développé simule plusieurs configurations de populations, notamment une population unique d'environ mille individus ainsi que des scénarios de fragmentation en plusieurs sous-populations plus petites. L'objectif est d'étudier l'impact de ces structures démographiques sur la capacité d'adaptation des populations et leur survie à long terme.

La simulation prend en compte différents phénomènes biologiques tels que l'apparition de mutations bénéfiques ou délétères, l'adaptation climatique polygénique, l'émergence d'un gène d'immunité ou encore la simulation d'une épidémie. Les simulations génèrent des jeux de données comportant plusieurs milliers de lignes décrivant l'évolution des populations au cours du temps.

Les données produites sont ensuite exportées au format CSV et analysées avec Python dans un environnement Jupyter Notebook. Une analyse exploratoire des données (Exploratory Data Analysis) est réalisée à l'aide des bibliothèques Pandas et Matplotlib, comprenant le nettoyage et le traitement des données, le calcul de statistiques descriptives et la visualisation de séries temporelles afin de comparer les différents scénarios simulés.

Ce projet permet notamment d'étudier l'impact de la fragmentation des populations sur la dérive génétique, la charge mutationnelle, la fitness moyenne et la capacité d'adaptation d'une espèce face à des contraintes environnementales.

Technologies utilisées : SLiM, Python, Pandas, Matplotlib, Jupyter Notebook, analyse de données.

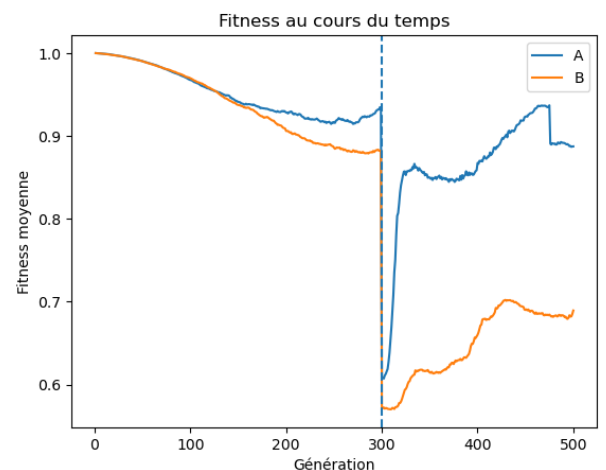


Fitness moyenne

```
plt.figure()
plt.plot(df_A["gen"], df_A["meanFitness"], label="1000")

df_B_fit = df_B.groupby("gen")["meanFitness"].mean().reset_index()
plt.plot(df_B_fit["gen"], df_B_fit["meanFitness"], label="10 de 100")

plt.axvline(300, linestyle="--")
plt.xlabel("Génération")
plt.ylabel("Fitness moyenne")
plt.title("Fitness au cours du temps")
plt.legend()
plt.show()
```



Projet Machine Learning – Classification d’images

Dans un second projet orienté machine learning, j’ai travaillé sur un problème de classification d’images visant à distinguer différentes catégories d’animaux à partir de caractéristiques visuelles. L’un des cas étudiés portait notamment sur la distinction entre images de chats et de renards, un problème intéressant en raison des similarités morphologiques entre certaines caractéristiques de ces espèces.

Le projet consistait à extraire différentes caractéristiques visuelles des images, telles que la forme, la couleur ou encore certaines textures, afin de construire un ensemble de variables exploitables par des algorithmes d’apprentissage automatique.

Plusieurs méthodes de classification ont été expérimentées, notamment les algorithmes K-Nearest Neighbors (KNN), Support Vector Machines (SVM) et arbres de décision. Les performances des modèles ont été comparées à l’aide de techniques de validation croisée afin d’évaluer leur capacité de généralisation sur des données non vues.

Ce projet m’a permis de découvrir les différentes étapes d’un pipeline de machine learning, depuis la préparation des données et l’extraction de caractéristiques jusqu’à l’entraînement et l’évaluation de modèles de classification.

Technologies utilisées : Python, machine learning, analyse de données.

Projet d’algorithmique – Compression de données avec OCaml

Dans le cadre d’un travail pratique d’algorithmique réalisé en OCaml, j’ai implémenté un algorithme de compression de données basé sur le codage de Huffman. Ce projet consistait à concevoir une structure d’arbre permettant de représenter un système de codage optimal à partir des fréquences d’apparition des caractères dans un texte.

L’implémentation a impliqué plusieurs étapes : calcul des fréquences des symboles, construction progressive de l’arbre de Huffman, génération du dictionnaire de codes binaires et mise en place des fonctions de compression et de décompression. Ce travail m’a permis de mettre en pratique des notions fondamentales d’algorithmique et de structures de données, notamment la manipulation d’arbres et la conception d’algorithmes efficaces.

Ce projet m’a également permis de renforcer mes compétences en programmation fonctionnelle avec OCaml, ainsi qu’en conception et analyse d’algorithmes.

Technologies utilisées : OCaml, algorithmique, structures de données (arbres).

```
(* Question 8
La fonction decodage_texte parcourt l'arbre de Huffman en suivant le mot binaire est par bit.
à l'issue d'un :
- on descend d'un niveau dans l'arbre,
- on effectue une opération conditionnelle (descendre sur la gauche et prendre un fillet).
La complexité est donc linéaire en la longueur du mot, soit O(N) où N est la longueur du mot binaire. Dans le cas d'un mot valide, la profondeur de la feuille représentant le caractère à décoder
ne peut donc aussi être que la complexité est proportionnelle à la profondeur du caractère dans l'arbre. Comme la profondeur maximale est liée à la hauteur de l'arbre, on a donc la pire cas : O(H), où H est la hauteur de l'arbre. Et comme pour "code_char", si
l'arbre est équilibré, H appartient à O(log N) donc "N" la nombre de caractères distincts.")
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

let exemple = ['Y'; 'Y'; 'Y'; 'Y'; 'Y'; 'Y'; 'Y'; 'Y'; 'Y'; 'Y']

let () =
  print_endline "-----";
  print_endline "Test question 8 (*)";
  let code = code_texte exemple in
    print_endline "Code du texte : ";
    List.iter (print_endline "%d") code;
    print_endline ();
  end

(* Projet 10 : comment expliquer qu'il existe du plus au moins 101 que les 10 premiers bits d'un mot codent un caractère dans un arbre de Huffman ? *)
(* Question 10
Chaque caractère est représenté par un chemin unique depuis la racine jusqu'à une feuille.
Cela signifie que :
- si un mot n'est qu'une séquence de caractères,
- alors il se peut arriver qu'un seul entier n'est pas les 10 premiers bits de n codent exactement le code binaire d'un caractère.
En effet, dès que la première des lettres est une feuille, on s'arrête : on ne peut pas continuer à descendre plus de bits pour obtenir un autre caractère (sans partager l'arête). Cette propriété rend le décodage sans ambiguïté possible, car on sait qu'en lisant
les bits de gauche à droite, on trouve exactement un caractère à la fois, et on peut reprendre la lecture après ce code. Donc, pour tout mot n et tout arbre de Huffman h, il existe au plus un entier k tel que les k premiers bits de n constituent le code binaire
d'un caractère.")
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 11 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 12 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 13 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 14 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 15 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 16 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 17 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 18 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 19 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 20 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 21 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 22 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 23 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 24 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 25 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 26 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 27 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 28 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 29 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 30 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 31 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 32 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 33 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 34 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 35 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 36 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 37 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 38 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 39 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 40 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 41 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 42 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 43 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 44 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 45 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 46 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 47 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 48 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 49 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 50 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 51 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 52 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 53 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 54 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 55 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 56 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 57 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 58 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 59 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 60 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 61 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 62 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 63 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 64 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 65 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 66 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 67 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 68 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 69 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 70 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 71 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 72 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 73 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 74 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 75 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 76 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 77 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 78 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 79 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 80 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 81 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 82 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 83 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 84 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 85 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 86 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 87 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 88 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 89 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 90 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 91 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 92 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 93 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 94 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 95 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 96 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 97 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 98 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 99 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end

(* Question 100 *)
let rec decodage_texte t =
  match t with
  | [] -> []
  | c::cst -> code_char c::decodage_texte t
end
```

```
-----
Test Question 2
Taille de l'arbre t : 6
-----
Test Question 3
Contient 'a' : true
Contient 'x' : false
-----
Test Question 184
Code de 'n' : 0 0 0
Code de 'f' : 0 0 1
Code de 'a' : 0 1
Code de 's' : 1 0
Code de 'i' : 1 1 0
Code de 't' : 1 1 1
-----
Test Question 6
Reconnait 000 : true
Reconnait 001 : true
Reconnait 01 : true
Reconnait 11 : false
Reconnait 0001 : false
Reconnait 111 : true
-----
Test Question 7
Le mot 000 decode 'n'
Le mot 01 decode 'a'
Le mot 111 decode 't'
Erreur pour 11 : mot non reconnu : terminé trop tôt
Erreur pour 0001 : mot non reconnu : trop long
-----
Test Question 9
Code du texte : 10011111101000101110111
-----
Test Question 11812
Texte décodé : satisfait
-----
Test Question 20
Tas de Braun : 1; 3; 5; 7
-----
Test Question 21
Tas après insertion de 3 : 2; 6; 8; 7; 3; 4
-----
Test Question 23
Dernier élément : 3
Tas restant : 1; 4; 2
-----
Test Question 25
Fusion : 1; 2; 4; 3; 5
-----
BONUS : Accélération du codage et Lecture/écriture de fichiers texte
Encodage avec arbre_test dans code.txt
```